Real Time Sentiment Analysis with CUDA

Samuel Bush, Candidate
George Rudolph, Advisor

Utah Valley University, Computer Science Department
sam.bush@uvu.edu

Abstract. This project studies real time sentiment analysis on streamed data that is processed on CUDA GPUs instead of CPUs. There are several purposes of this project: The main goal was a vehicle for learning how to program GPUs and general-purpose GPU programming with CUDA. A second goal was to learn about the field of sentiment analysis. A third goal was to explore whether a sentiment analysis algorithm could be improved using CUDA versus CPU. Real-time sentiment analysis is computationally heavy, and it is difficult to process large amounts of streamed data, such as a Twitter feed, with a quick turn around time. Results are that CUDA has a steep learning curve, CUDA should be used for specific use cases instead of general purpose programming and that the nature of sentiment analysis does not lend itself well to GPU processing. In conclusion, the recommendation of this project is to use Pytorch or TensorFlow instead of using CUDA directly. This project amounted 1200 of lines of code.

Keywords: Sentiment Analysis, CUDA, cuDNN, LSTM Neural Network, GPGPU

1 Introduction

The problem that this project is tackling is a two part problem about whether a successful implementation of sentiment analysis on streamed Twitter data is possible and whether or not performance of the sentiment analysis could be improved by using Compute Unified Device Architecture (CUDA) cores instead of CPU cores. This is interesting because if using CUDA cores for general purpose programming is successful, CUDA could be applied to other algorithms to improve performance. Parallel programming is the key to improving performance for repetitive tasks, instead of simply increasing clock speeds. There are inherent problems with increasing CPU performance and clock speeds such as heat limitations and Moore's law. Some would argue that Moore's law is obsolete and that CPU performance will not improve as quickly as it once did[1]. The major results of this project were that CUDA implementation is used mostly for simplistic calculations, and not well suited for general purpose programming. CUDA should not be used for general purpose programming, only on more specific computationally heavy use cases, that rely heavily on pre-processed static data. Extensive experience and knowledge of CUDA is necessary for this project

and CUDA could not be implemented in the way this project first intended it to be used. This project recommends using Tensor cores instead for future implementation.

Sentiment analysis is a way of measuring the relationship between two variables and assessing those variables to categorize them into specific sentiments. The goal is to form a relationship between these two variables and to determine which sentiment fits the variable best: whether the sentiment is good or bad. "Sentiment analysis is the process of computationally identifying and categorizing opinions expressed in a piece of text."[2] It is often used to determine whether the author of a particular piece of text or, in this case, the tweet is positive, negative or neutral. Sentiment analysis could be used to determine if the authors intentions are also truthful by mining contextual clues in the writing. One would do this by finding key aspects related to the subject and the underlying intentions and reactions concerning those subjects. Misinformation can become widespread rapidly and can result in the truth becoming hard to identify. It is important to identify misinformation as quickly as possible. In fact, tech giants such as Facebook have had their policies related to political ads questioned and often blamed for failing to fight misinformation. [3] Sentiment analysis can be done on any commercially available computer, however, in order to get real-time sentiment analysis you will need to achieve near instantaneous results. This project explores whether a regular consumer desktop could realistically perform real-time fact checking using single threaded CPU programming or CUDA Cores.

The purposes of this project are as follows:

1. To build a neural network to do sentiment analysis
2. To achieve accurate results on streaming data in real time
3. To learn CUDA

The intent of this project was to build a neural network and perform sentiment analysis using data mined from the Twitter feed in as close to real time as possible. This project's developer wanted to harness the power of the GPU to achieve as close to real-time sentiment analysis equivalent performance as possible. The goal was to be able to analyze a tweet on the Twitter feed within one second after the Tweet had been posted and collected. The program would then post a sentiment score where a negative score means a negative sentiment, a positive score means a positive sentiment and, a neutral tweet would have a score of zero. The more positive or negative the score, the stronger the sentiment. GPUs are commonly used for neural networks. Software developers can get better throughput when using the GPU for parallel programming because neural networks and sentiment analysis are embarrassingly parallel. This made sentiment analysis a good candidate for CUDA programming - by using CUDA Cores the time needed to process large amounts of data could be reduced since a GPU has thousands of more cores than any one consumer grade CPU.

The other main goal for this project was to learn CUDA. The author chose not to use TensorFlow 2.0 + Keras or PyTorch for the project. CUDA is an

API for parallel computing using Nvidia GPUs. The author believed that using CUDA directly would have offered better performance. CUDA and Nvidia GPUs are used in many areas of deep learning and machine learning. Parallel programming is the foundation for some of the fastest computers in the world and CUDA provides everything necessary to develop GPU-accelerated parallel applications. [4] TensorFlow and other frameworks use CUDA libraries and tool-kits for running on Nvidia GPUs. The CUDA library is so important to the training of the deep learning frameworks that all of the frameworks using a given version of cuDNN have essentially the same performance numbers for equivalent use cases. There are plenty of resources on the internet that could have provided the author with the code and instructions for running it on AWS in less than an hour using TensorFlow. However, the author wanted to learn to interface with the GPU directly to get the most out of the GPU cores that the application will use. The author had done work on a previous project that performed simple sentiment analysis on previously collected tweets. An aspiration for the project was to have the program learn unsupervised from streamed data how to analyze and respond to tweets. They were hoping to use an LSTM (Long Short Term Memory) neural network in order to achieve this goal. LSTM neural networks are great for machine learning and sentiment analysis because they are capable of learning long-term dependencies. This would be important when working with streamed data as sentiments change over time.

What the author learned was that sentiment analysis is well suited for even a single core processor with Twitter data. This paper details the reasons for using Pytorch and TensorFlow instead of learning and using CUDA for development. What makes this project complex is that it uses two different programming languages and must communicate and interact between programs. Also CUDA is not as well known as many other technologies so there are not as many forums and resources out there to help guide developers to the right solutions. Having separate programs interact with each other asynchronously and working with dynamic streaming data is a challenge in itself with most programs. This project amounted 1200 of lines of code.

The rest of the paper is laid out as follows. Section 2 is a literature review. Section 3 describes the design and implementation of the system. Section 4 explains ideas that are needed to understand the project along with tools and setup information. Section 5 presents results and gives the technical assessment of the project. Section 6 discusses aspirations for the project after this graduate project is completed. Finally, section 7 is the conclusion. Appendix 8 is the user manual for this project.

2 Related Works

Social media is a useful platform to share health-related information due to its vast reach. This makes it a good candidate for public-health monitoring tasks. In "Sentiment Analysis: Concept, Analysis and Applications" they study the problem of extraction of Adverse-Drug-Reaction (ADR) mentions from social

media, particularly from Twitter. Medical information extraction from social media is challenging, mainly due to short and highly informal nature of text, as compared to more technical and formal medical reports. The word embedding based features are trained on a large domain-specific tweet corpus. The problem with that approach is its dependency on hand-crafted features, which is time and effort consuming. A Long-Short-Term-Memory (LSTM) network based model is proposed to get around this problem. Instead of using hand-designed features, word embedding based features are passed to a Bi-directional LSTM model which is trained to generate a sequence of labels, given the input word sequence, in doing so better results are achieved, surpassing Conditional Random Field-based ADRMine results.[2] This research pertains to this project in that the project is also mining data from Twitter in the hopes of gleaning useful information from data mining social media as well. This was useful information as the CUDA project sought to implement a similar LSTM neural network to generate labels by itself instead of providing them.

In "Influence of Fake News in Twitter During the 2016 US Presidential Election," the author used a dataset of 171 million tweets in the five months preceding the election day to identify 30 million tweets, from 2.2 million users, which contain a link to news outlets. Based on a classification of news outlets curated by www.opensources.com, they found out that 25% of these tweets spread either fake or extremely biased news. [5] This was relevant to this project in emphasizing the need to rely on machine learning algorithms sentiment analysis on social media sources. It's an extremely difficult task, if not impossible for human moderators to stop misinformation.

In "Automated Fake News Detection in the Age of Digital Libraries." The authors construct a fake news detection system. They were concerned about the free production and consumption of large amounts of data provided by the internet. Fact-checking systems powered by human efforts are not enough to question the credibility of the information provided, or to prevent its rapid dissemination. Considering that libraries are undergoing digitization processes all over the world and are providing digital media to their users, it is very likely that unverified digital content will be served by world's libraries. The solution is to develop automated mechanisms that can check the credibility of digital content served in libraries without manual validation. [3] How this pertains to this project was that it stressed the need for the LSTM neural network to generate its own labels as the vast amounts of data it would have to sift through is constantly changing, meaning that it would always have to be training its connections and labels.

In "Deep learning for Pharmacovigilance: Recurrent Neural Network Architectures for Labeling Adverse Drug Reactions in Twitter Posts," the authors also use social media for adverse drug reaction (ADR) identification. The approach they use is using a recurrent neural network (RNN) model that labels words in an input sequence with ADR membership tags. Their model required no task-specific feature engineering, suggesting generalizability to additional sequence-labeling tasks. Learning curve analysis showed that their model reached optimal performance with fewer training examples than the other models. Detection per-

formance in social media is significantly improved by using a contextually aware model and word embeddings formed from large, unlabeled data sets. The approach reduces manual data-labeling requirements and is scalable to large social media data sets. [6] This provides an interesting approach to reduce manual data-labeling with a contextually aware model. Ultimately, however, the author of the CUDA project would like to eliminate manual data-labeling as much as possible and have the LSTM neural network form as many of its own labels as possible.

In “Pharmacovigilance from Social Media: Mining Adverse Drug Reaction Mentions Using Sequence Labeling with Word Embedding Cluster Features,” the authors recreate the same experiment as the previous article. They conclude that it is possible to extract complex medical concepts, with relatively high performance, from informal, user-generated content by using word cluster features to improve extraction performance and diminishing the need for annotated training data sets when performing sentiment analysis on large volumes of unlabeled data. [7] This was insightful in order to understand exactly how they would approach extracting data from Tweets, although they implemented a co-occurrence matrix instead of a full on word cluster feature.

In “Novel Machine Learning Approach for Sentiment Analysis of Real Time Twitter Data with Apache Flume.”, the authors are storing the Twitter Streaming Data into HDFS distributed file system of Hadoop by using Flume and then extracting with Apache Hive. A novel approach based on hybrid Naïve Bayes and Decision Tree Algorithms are used to enhance the performance of sentiment analysis of streaming Twitter data. The implemented research approach achieved an accuracy of 86.44% in comparison to 81.11% for Naïve Bayes Classifier.[8] It was interesting to see this hybrid approach with sentiment analysis, as well as their unique approach to storing and using streamed data, which gave some ideas for future implementation in this current project.

In “Convolutional Neural Networks for Sentence Classification” the authors report on a series of experiments with convolutional neural networks (CNN) trained on top of pre-trained word vectors for sentence-level classification tasks. They show that a simple CNN with little hyperparameter tuning and static vectors achieves excellent results on multiple benchmarks on 4 out of 7 tasks, which include sentiment analysis and question classification. They also propose some modifications to architecture to allow for the use of task-specific and static vectors. [9] Ultimately after reading this article it became clear that convolutional neural networks were not a great fit for the CUDA project which uses largely untrained dynamic data. Although it was interesting to see the solutions proposed in the article for increasing the performance of convolutional neural networks.

In "Sequential Short-Text Classification with Recurrent and Convolutional Neural Networks" the author discusses recent approaches based on artificial neural networks (ANNs) for short-text classification. Many short texts occur in sequences (e.g., sentences in a document or utterances in a dialog), and most existing ANN-based systems do not leverage the preceding short texts when classifying a subsequent one. The author presents a model based on recurrent neural

networks and convolutional neural networks that incorporates preceding short texts.[10] This was essential to understand as the data in the CUDA project was mostly comprised of short texts. The data was largely collected from Twitter where it is not uncommon to see tweets with only just a few words. It can be concluded that recurrent neural networks are a powerful sequence learning architecture and are proven capable of transforming any input sequence into any finite, discrete output sequence.

In "Sequence Transduction with Recurrent Neural Networks" the author stresses the need for natural language processing techniques by machine learning algorithms as human review of social media data is infeasible due to data quantity. Other challenges of social media include informal vocabulary and irregular grammar, which challenge natural language processing methods. The author's best-performing RNN model used pretrained word embeddings created from a large, non-domain-specific Twitter data set. Feature analysis indicates that semantic information in pre-trained word embeddings boosted sensitivity and, combined with contextual awareness captured in the Recurrent Neural Networks precision.[11] This is similar to the CUDA project except that instead of using a recomposed Twitter data set, streamed Twitter data is used and was not able to be pre-trained word embeddings.

In "Long Short-Term Memory" the author discusses the problems with recurrent network algorithms learning to store information over extended time intervals by recurrent backpropagation. This process is time consuming, because of insufficient, decaying error backflow. The author addresses the problem by introducing a novel, efficient, gradient based method called long short-term memory. Multiplicative gate units learn to open and close access to the constant error flow. long short-term memory also solves complex, artificial long-time-lag tasks that have never been solved by previous recurrent network algorithms.[12] This article was essential to understand LSTM and helped when deciding the best neural network architecture to use for streamed Twitter data.

In "Parallel Implementation of Big Data Pre-Processing Algorithms for Sentiment Analysis of Social Networking Data," the author discusses the need for parallel pre-processing of social networking data. Due to the huge volume, the highly unstructured nature of the data and the enormous rate at which it is being generated, parallel implementations of pre-processing algorithms become important. Pre-processing includes string to vector conversion, elimination of trivial words and symbols if any, frequency mapping etc. They implement and compare parallel algorithms for pre-processing of social networking data (Twitter feeds) based on General-Purpose Computing on Graphics Processing Units (GPGPU) (NVidia CUDA C) Architectures. They also compare and contrast the benefits and drawbacks of GPGPU in using parallel algorithms for effective and timely pre-processing of data. [13] This was one approach that was considered when implementing GPGPU into the CUDA project. In retrospect, this would have been more effective and would have increased the throughput of the CUDA program.

In "Parallel Computing Experiences with CUDA" the author discusses how CUDA programming model provides a straightforward means of describing inherently parallel computation on massively parallel problems. This article surveys experiences gained in applying CUDA to a diverse set of problems and the parallel speedups over sequential codes running on traditional CPU architectures attained by executing key computations on the GPU.[14] This was important because it helped me to formulate what kind of computational performance increase should be expected when using CUDA. Although ultimately the CUDA project was not well suited for parallel computations because of it's dynamic nature and the part that needed to be parallel heavily relied on string comparisons and not simple computations.

In "High Performance Twitter Sentiment Analysis Using CUDA Based Distance Kernel on GPUs," sentiment analysis techniques are widely used for extracting feelings of users in different domains such as social media content, surveys, and user reviews. The high dimensionality of the feature space creates a serious problem in terms of time and performance for sentiment analysis. To cope with this problem, the author used sentiment analysis techniques using NVIDIA's CUDA technology. They employed their CUDA-based distance kernel implementation for k-NN classification. [15] While the CUDA project did not use k-NN for sentiment analysis, NVIDIA's CUDA technology was implemented to speed up the performance of the sentiment analysis program.

In "Feature-Based Sentimental Analysis on Public Attention towards COVID-19 Using CUDA-SADBM Classification Model" the author constructs a model for sentiment analysis of COVID-19 data on social media, a unique classifier named the Sentimental Database Miner algorithm (SADBM) is used to categorize the opinions and parallel processing, and is applied on the data collected from various online social media websites like Twitter, Facebook, and LinkedIn. The accuracy of their model is validated with trained data and compared with basic classifiers, such as logistic regression and decision tree. Their algorithm is executed on CPU as well as GPU and calculated the acceleration ratio of the model.[16] This article was the most similar to this paper's CUDA project, but instead of using SADBM, the Pointwise Mutual Information calculation was used.

3 Software Architecture and Implementation

You can see here in Figure 1 that this project would be split into several different programs. The first program would interface with the Twitter API to retrieve data from a live Twitter feed and then parse those tweets into JSON (JavaScript Object Notation). That program would then pipe the parsed JSON into the CUDA program via std out in a string. The CUDA program would then perform sentiment analysis and pipe the processed data back to the Python program and the Python program would print the output of the processed tweets. The intention was to have the CUDA program running in streaming mode and be listening so that it wouldn't have to close out every time it gets a new task.

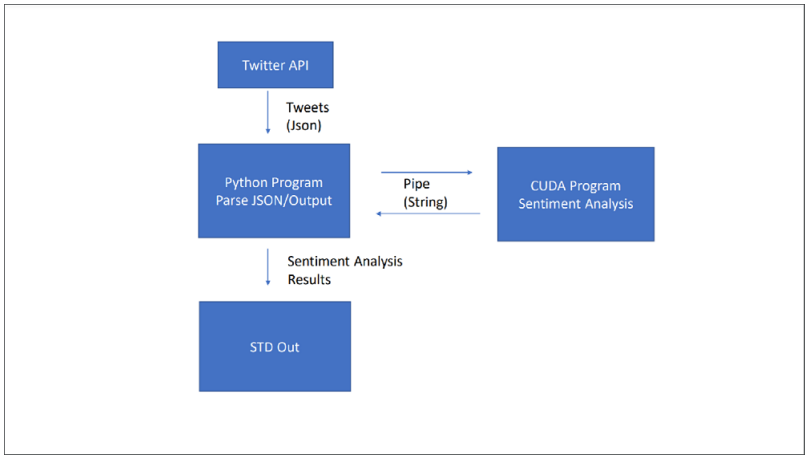


Fig. 1. High Level Project Diagram

This is the reference of how sentiment analysis and semantic orientation will be handled in the CUDA project as you can see here in Figure 2. The tweet will be passed in to the program via STD in the form of a string by the Python project. The program would then tokenize and push the string to an array. The string would consist of all the tweets and separated by break lines. Once the tweets have been tokenized and pushed into an array for processing the next step would be to build a co-occurrence matrix. Building a co-occurrence matrix is essential because in order to find the difference in distance from one word to another we need to first figure out the relation of the word to all of the words in the tweet. How this is done is you would take the first word and then make pairs with all the other words in the same tweet and so on for the next word until the entire tweet is put into a co-occurrence matrix. You don't want any duplicates and when you have built all possible co-occurrences in the matrix you move onto the next tweet and build the co-occurrence matrix for that tweet. This is essential because it will be used for determining the distance of the word in relation to other words in the tweet and if there are a lot of bad words then you can associate that those words have a negative association. Once you have done that, you then calculate the probability of the term occurrence. This is where you take the number of times that the single word appears in the entire data set and you divide it by the number of tweets and that would give you the probability of that tweet occurring in any given tweet in your data set. This is also important for calculating the semantic orientation. Once you have those calculations made, you can then proceed to the pointwise mutual information calculation which is used to calculate the value of negativity and positivity of any given word and the distance of that negativity or positivity to the bad or good words. The Pointwise Mutual Information calculation is the following:

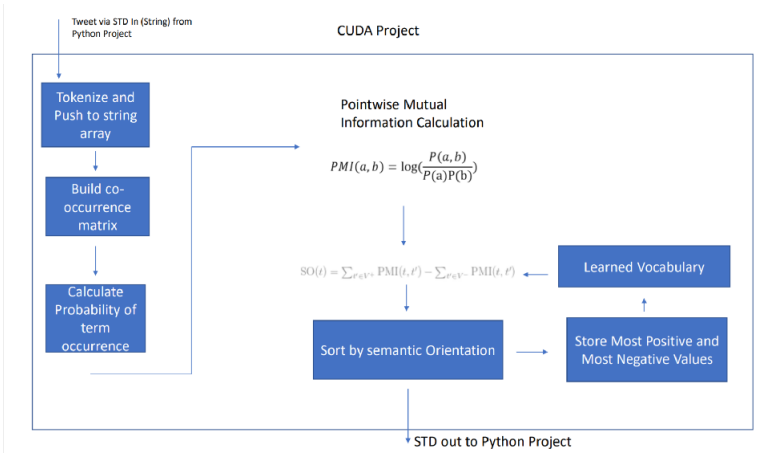


Fig. 2. CUDA Program Diagram

$$PMI(t_1, t_2) = log(\frac{P(t_1 \wedge t_2)}{P(t_1) * P(t_2)})$$

Above is the Pointwise Mutual Information calculation. In order to get the Pointwise Mutual Information value for t you would need to get the two terms (t1 and t2). This goes back to our co-occurrence matrix that we were creating. This is calculated for every value and every set in our co-occurrence matrix. What you would need is to take the probability of term 1 and term 2 that was calculated earlier and this is the probability of the term occurring in any given tweet. Then, you would divide that by the product of the probability of term 1 and term 2. Once you have done that you would then take the log of the previous result and then you will get the Pointwise Mutual Information value for each of the term co-occurrences in the co-occurrence matrix. Then the next thing that you would do is calculate the semantic orientation.

$$SO(t) = \sum_{t\prime \in V+} PMI(t, t\prime) - \sum_{t\prime \in V-} PMI(t, t\prime)$$

Above is the formula used to calculate the semantic orientation. To calculate the semantic orientation, you take the sum of all the values after calculating the PMI for all of items in the t-prime vector.

4 Methods

The following are methods and concepts that are used throughout the project. The plan was to stream and capture incoming tweets pertaining to a particular

hashtag or keyword and analyzed the sentiment of these tweets, then perform sentiment analysis in two different environments one with a single threaded program and the other in a CUDA program and compare the performance.

4.1 Tools and Technology

The equipment used was a NVIDIA GeForce GTX 1080 Graphics Card as well as an Intel i7-8700K to compare performance of the sentiment analysis program on. The single threaded version of the program was built in C++. NVIDIA CUDA and cuDNN (CUDA Neural Network Library) was used to build the parallel version of the program that would run on the GPU. Python was used to build the program that would pre-process data as well as handle the Twitter API. Other tools used are Windows 10, NVIDIA CUDA Developer Toolkit, and Visual Studio 2019, and Anaconda Python and NVIDIA NSIGHT for CUDA debugging.

4.2 System Setup

There are many approaches to implementing sentiment analysis including linear regression, support vector machines and deep learning. This project attempts the deep learning approach by implementing a neural network. This project uses GPU in comparison to CPU, since you can get better throughput when using the GPU for parallel programming because neural networks and sentiment analysis are embarrassingly parallel and GPUs have so many more cores than CPUs. GPUs were originally developed for gaming uses; it's specialized for matrix operations. GPUs also have a lower clock speed than a CPU but what they lack in speed they make up for in number of transistors which translates to more cores. In modern data processing and research, processing large amounts of data is very critical for a lot of researchers and companies. However, processing a huge amount of data requires a significant time investment that can cause a significant slowdown in performance. Performing linear regression in parallel is one approach to make data processing even faster. NVIDIA has developed a way to harness the computational power of the GPU's transistors which will give us thousands of more cores to work with for parallel processing than just a single CPU using CUDA. Using the GPU's transistors as computational cores will help us to improve performance even more. This puts the GPU at a much greater advantage over the CPU for parallel programming since the GPU has thousands more cores and is optimized for matrix operations. The plan was to use a recurrent neural network because it's better than a convolutional neural network for text and speech analysis, when working with larger data-sets over time. This would have been especially important because Twitter constantly produces new data so the neural network needed to keep learning new things as well as retain old information. The type of recurrent neural network that was intended to be implemented was a LSTM variation of neural network that would be able to memorize information for longer periods of time with a higher

degree of accuracy. [17] Due to time constraints, this project implements a simple convolutional neural network architecture instead. Lastly, this project used the Pointwise Mutual Information calculation to get the semantic calculation for each word. Relu would have been a better choice as the activation function for the neural network because Relu is much easier to train neural networks than sigmoid. [8] With more time, Relu would have be implemented to determine which words to keep after performing sentiment analysis since it would be on a scale from -1 to 1 instead of relative to other tweets sentiment calculation. That would have made it easy to determine quickly whether a tweet was positive or negative, instead of taking into account the other tweets relative values.

The next steps would be to conduct an experiment by creating a sentiment analysis program that would run on the GPU in parallel. Then, bench-mark the linear regression with CUDA on the GTX 1080 to see how much better performance you could get out of linear regression vs. non parallel, parallel, or parallel with GPU.

4.3 Collecting Data

Social media has almost become synonymous with "big data" due to the sheer amount of user-generated content. Mining this rich data can prove unprecedented ways to keep a pulse on opinions, trends, and public sentiment. Furthermore, every generation is spending even more time on social media than their predecessors. This means that social media data will become even more relevant for marketing, branding, and business. While there are many popular social media platforms out there, Twitter is the classic entry point for practicing machine learning. With Twitter data, you get an interesting blend of data (tweet contents) and meta-data (location, hashtags, users, re-tweets, etc.) that open nearly endless paths for analysis. How data was collected from Twitter was by signing up for the Twitter developer account and then getting familiar with the Twitter API and learning how to send data to the application using API commands, tweepy (a Python library for accessing the twitter API) and the private keys for the project's Twitter account.

4.4 Text Pre-processing

There's a lot of information that you can gather from a tweet. We only want the text itself for the application to work. Part of the pre-processing was to tokenize the tweets. This meant breaking down the text into smaller units called tokens, which consist of just a few words or phrases. While this is a well understood problem with several out-of-the-box solutions from popular libraries, Twitter data poses some challenges because of the nature of the language, for example heavy use of phrases, and unique abbreviation of words. A custom tokenization method specifically designed for Twitter/social media was used and added to so that it would also tokenize emojis and Twitter jargon as well.

4.5 Term Frequencies

Counting term frequencies was necessary for sentiment analysis because there was a large amount of data from the incoming streamed tweets to sift through, so it was necessary to consider only the most frequent terms in the analysis. Term frequencies also does have some other insights about the data that were interesting to view too. There were some considerations when determining the most frequent words. For example, the word “the” in English is probably the most commonly used word, however, it does not provide much insight. There are many words like this in the English language that needed to be considered when determining what would be included in the most frequently used words list. A dictionary containing these words was compiled as well as an imported one from the internet to filter out from the most common words. There was a need to include a lot of non-English words too since the program was collecting data in all languages even though it was streaming tweets specifically with the "English" metadata in the header. The tweets that were collected were mostly English but there were some that were mixed language and had non-English words or names in them.

4.6 Term Co-Occurrences

Term co-occurrences are used later when we want to learn more specifically about the sentiment of those occurrences. The term co-occurrences frequency number in the terms that occur together was essential to the semantic calculation. This is mainly because the context gives us a better insight about the meaning of a term, supporting applications such as word disambiguation or semantic similarity. Term co-occurrences are used to build our neural network using a co-occurrence matrix. This is then later used to perform matrix calculations that will help to determine the semantic calculation. The more tweets gathered with different term co-occurrences, the larger the neural network gets because those values get stored and expanded on. You could calculate the number of new neural connections by using the formula n * n-1 where n is the number of words in a tweet. This is a pretty common equation to figure out how many cables you would need to create a mesh network between computers.

4.7 Sentiment Analysis

The technique used was based off of Peter Turney’s paper “Thumbs Up or Thumbs Down?" He defined Semantic Orientation of a word was the difference in associations with positive and negative words. [18] This project uses the Pointwise Mutual Information calculation that can be found in Turney’s paper. Furthermore, in Turney’s paper the semantic orientation was calculated against the words excellent and poor. The vocabulary was extended to 4,000 positive and 4,000 negative words, phrases, and emojis commonly used in Twitter. Those were added as a text file and then the CUDA or single threaded program will load them into memory and push them to a vector that will be used to compare

whether a word is positive or negative. Semantic orientation is calculated with the following with V+ being positive vocabulary and V- being negative.

$$SO(t) = \sum_{t\prime \in V+} PMI(t, t\prime) - \sum_{t\prime \in V-} PMI(t, t\prime)$$

5 Results

Pertaining to the criteria of making a neural network to do sentiment analysis it was successful. There was a successful instance of the sentiment analysis happening on a single thread in C++. An instance of LSTM neural network was not implemented to learn sentiments. That being said despite not using a LSTM neural network and providing the sentiments, it performed well. Below are some examples of a positive tweet, negative tweet and neutral tweet.

5.1 Example Tweets

Positive Tweet

```
RT @scotgov: The vaccine is our best defence against #coronavirus.
To protect yourself and everyone you love, it's important to get your...

Top Positive
everyone = 2105.52
tests = 1885.36
vaccine = 1816.73

Top Negative
Amsterdam = -20241.1
people = -16564
COVID = -15896

Tweet Score: 481.871
Time Elapsed: 442468 Microseconds
```

You can see here in the positive tweet that if you read it the overall sentiment of the tweet is positive. As you can see the tweet score is positive indicating that the sentiment for the score is positive. The running top 3 positive words are printed and you can see that "vaccine," which is included in this tweet, is one of the top three positive words according to the sentiment analysis. The other two words are not random as well but they are missing the tweets so it doesn't provide much context. The tweets that were seen mentioning "free tests for everyone" would be a positive sentiment.

Negative Tweet

```
RT @Ruptly: Police use dogs, batons in violent clashes with anti-lockdown protesters
in Amsterdam #COVID19 #coronavirus #Amsterdam #Nether...

Top Positive
everyone = 2055.15
tests = 1832.35
vaccine = 1777.53

Top Negative
Amsterdam = -20241.1
people = -16080.9
COVID = -15550.3

Tweet Score: -239.631
Time Elapsed: 419303 Microseconds
```

This is an example of a negative tweet. You can see from the tweet that the sentiment is extremely negative with words including violent, clashes etc.. so the score here should be pretty negative and it is. You can see the top three negative are "Amsterdam," where the violence is happening that people are talking about, "people" (In the context of how many have been infected or died) and "COVID."

Neutral Tweet

```
@MenaceSocietyUT @tedcruz Never Forget.....
https://t.co/bosNP3U6nC

Top Positive
everyone = 2144.49
tests = 1885.36
vaccine = 1816.73

Top Negative
Amsterdam = -20389.5
people = -16564
COVID = -16013.6

Tweet Score: 0
Time Elapsed: 134804 Microseconds
```

This tweet could have a positive or negative sentiment but it's hard to tell because there's not enough information. It looks like there's a link to a picture

or a tweet but because the program doesn't analyze photos or links the context is missing and therefore this tweet has a neutral sentiment.

5.2 Streaming Data

This project's second goal was to achieve accurate results on streaming data in real time. This project was able to achieve the goal of processing most tweets under 1 second which was an achievement, although the project did not need to use CUDA in order to achieve this goal. Tweets are only a few characters long and this worked to an advantage. This project runs asynchronously, it listens and waits for a tweet then it sends it to the C++ program and waits for a response before listening again, and it can only process a few tweets a minute despite the less than a second sentiment analysis process time.

While the individual process time should be shorter with a CPU than it would be with slower GPU cores at first, as the neural network grows, and the amount of comparisons increases, you would notice the CPU times slowing down more than the CUDA times would. If the comparison was between which one would process one tweet quicker it would probably be CPU because when running CUDA programs you actually need to copy the data from the host (CPU) to the device (GPU) which requires quite a bit of overhead. Also, CPU cores are much faster than GPU cores. A better test to see whether CUDA or Single threaded program is more suitable would be to test the throughput over 10,000 tweets and see which one is quicker. What was found was that as the neural network grew the processing time grew as well. As the neural network develops you can see it taking more time to process the same tweets. You can see why the performance would be better suited with a CUDA program as opposed to a single core. The CUDA program would be able to perform each comparison on a different core making up for the overhead costs in the long run. In the table below you can see how much more time processing takes over the course of just 500 tweets. The time required increases substantially and the speed decreases about 22% over the course of just 500 tweets. If you were to test it out on 10,000 tweets you would see an even steeper increase in processing time.

Parameters		Results			
Tweet	Word Count	Tweet Score	Process Time	Difference	Percentage
2	6	-74.5111	391669	0	100%
88	6	-111.453	414305	22,636	0.94536%
204	6	-315.11	429590	37,921	0.911727%
530	6	-302.156	497774	106,105	0.786841%
13	22	-71.531	436596	0	100%
28	22	-86.2303	448397	11,801	0.97368180%
119	22	-136.728	502955	66,359	0.868061%
544	22	-181.373	547216	110,620	0.797849%

Table 1. Project Results

5.3 CUDA

Regarding the last goal of the project to learn CUDA, the author of this report learned a lot in the course of this project. The author learned some valuable lessons about GPGPU (General-Purpose Graphics Processing Unit) programming but, especially with working with CUDA.

One thing that was difficult was that the original program was written in Python which is completely different from CUDA. In order to make it into a CUDA project, it would first have to be made into a single thread C++ program because CUDA utilized C++ at its core. This required a thorough understanding of C++ and its utilities as well as Pointers. While there are some similarities between the Python, CUDA, and C++ the syntax and nuances are different and that made things more difficult than anticipated. Getting the Twitter API to interact directly with the C++ program is a pretty niche thing to implement in C++ and there's not a lot of C++ developers that work with Twitter API. As a consequence, there aren't a lot of diverse libraries or resources for processing JSON and Twitter API on CUDA or C++. It soon became clear that it would be easier just to have a separate program interact with the Twitter API and process the tweets in Python. After coming to that conclusion, it was decided to make part of the program in Python, which does have a lot of common tools and libraries for working with Twitter API and JSON, and then pipe that processed data to the C++ program. A separate C++ program was made for the parts that would eventually be written in CUDA and then worked on creating a CUDA application from it once it was working. This project was split up into 3 separate programs, the C++ single threaded program, the CUDA program that would each perform sentiment analysis separately, and the Python program that would process the tweets and parse the JSON data.

There was difficulty getting the programs to work together and eventually it was necessary to settle on piping a long string to the next program. The C++ program worked, but heavily utilized things such as dictionaries. It was also a challenge to train the machine with live data because it would constantly be changing and used dynamic data for most things. While implementing the CUDA program it was discovered that CUDA was not very good at handling dynamic data and that dictionaries don't work at all in CUDA; it is much better for you to use a static vector and then implement the CUDA on very small parts of the program instead of making the entire process parallel. There was an attempt to create an array of fixed length strings and copy the input strings to it, then copy the array to the GPU by passing the base address of the array of strings to the kernel and have each thread calculate the address to a given string by adding an offset based on its index to the base address. Once the data was copied to the GPU it would work with some data types but not with strings. String comparisons was one of the processes that was intended to run in parallel. It would have been very useful for comparing the co-occurrence matrix to the lists of good and bad words. After days trying to implement that, it was later found out that you also cannot use the C++ string type in a kernel because CUDA does not include a device version of the C++ String library that would be able run on

the GPU. Even if it was possible to use string in a kernel, it's not something you would want to implement because strings handle memory dynamically, which would be likely to be slow and difficult. That made the project more difficult because a lot of the repetitive tasks that would originally be made parallel tasks were string comparisons.

There were challenges especially when it came to troubleshooting and debugging the CUDA application since it needs to be run on the GPU and not the CPU. Debugging is an entirely separate thing that required using NVIDIA Nsight which was pretty challenging and unreliable at times. A lot of time was lost to debugging the debugger. At times, there was a need to reconfigure and reinstall NVIDIA Nsight just to get it to work. After some time the CUDA program was able to be reliably debugged. Another issue was taking data stored in the memory on the host and performing cudaMemcpy, which copies the memory from the location stored in the pointer of the host to the GPU VRAM or GPU memory. When copying over the data from the host to the device the pointer would get lost and point to some random data in the device. The program was able to do simple matrix calculations in CUDA but not with streamed Twitter data. It would work with simple integers but had difficulty with floats and other data types. The CUDA part of it worked, but not to the extent that it needed to. Although knowing exactly what the problem was and when it was happening, the cause of the problem went unresolved.

6 Further Implementation (Next Steps)

This section was made to describe how to go about continuing work on this project and some aspirations for the project in the future. This section will also detail some recommendations for others who embark on a similar project, some of the key things learned, and what could be done differently. An aspiration of this project was to have a really good improvement on performance from the single threaded C++ program to the CUDA program. It would be interesting to see how much longer it would take for one program to complete vs the other one to complete, to see if there really was much of a performance boost. If the CUDA portion of this project was functional and completed there would have been added another column in the results table: to compare single threaded vs CUDA performance boosted. It would be interesting to try it out on another GPU as well. There could have been another column for GTX 1080 and one for GTX 3090 to see if a more modern GPU would improve the performance.

Another idea that is worth exploring would be to implement the LSTM recurrent neural network so that the project could have persistence over time. This would imitate how a person learns things over time more accurately than a simple convolutional neural network would. Process takes longer as the neural network gets bigger. A future goal for the project would be to create a persistent neural network. In order to do this, the neural network would write to file new words that it learned the sentiments of. It would be interesting to see how it would learn over the course of a week. It would also be beneficial to implement

Relu as an activation function to determine which words to keep after performing sentiment analysis since it would be on a scale from -1 to 1 instead of relative to other tweets sentiment calculation. Words that were .999 positive or -.999 negative would be written to their respective files.

Another future implementation would be to get string comparisons to work in CUDA. A possible solution to getting around strings not working in CUDA could be to hash the string to a value and use that value to compare other hashed values.

Right now the programs are running asynchronously but in future implementations it would be nice to get it to run synchronously so that it could process even more tweets in less time since the Python program does not need to wait for a response from the CUDA or C++.

It would be interesting to see how the performance differed when using CUDA vs Pytorch or TensorFlow.

7 Conclusion

Sentiment analysis was rewarding to implement and eye opening. A lot can be learned from sentiment analysis. It's best to have an idea of what you are trying to learn about in order to get the best results. Sentiment analysis requires pre-processing in order to get from thousands of raw tweets to useful information. That pre-processing included included stripping out the meta-data, removing unusable characters, filtering out common English words without any sentiment such as "the," and parsing from JSON to plain text.

It was not possible to see what kind of performance boost could have been achieved by using an NVIDIA Graphics Card and CUDA Cores for general purpose programming instead of CPU cores. The sentiment analysis program had difficulty running in parallel with CUDA, because of the nature of streamed data. Streamed data is comprised of more dynamic data that makes it challenging to work with in CUDA and much more challenging to trouble shoot as well[6]. The sentiment analysis program was likely not a very good candidate for CUDA as CUDA works better with static integer vectors and does not work very well with dynamic data or string comparisons. This project recommends using Pytorch or TensorFlow as opposed to working directly with CUDA.

TensorFlow has a lot more tools built out than simple CUDA and you don't need to have knowledge of the hardware that you are running it on. TensorFlow handles that all for you. CUDA development is not very user friendly and there are still a lot of tools that need to be built out. It would seem that CUDA development is not intended for wide adoption like Pytorch or TensorFlow.

It is important to note that a high level of C++ knowledge is necessary for this project in order to get better performance. A more experienced C++ developer will have no problems, however, before embarking on future work with this project it is important to have a profound understanding of C++ and best practices. Spend time reviewing C++ and get familiar with it so that you will not have as much difficulty trouble shooting CUDA issues.

In conclusion, this project successfully achieved its goals which were to build a neural network to do sentiment analysis, achieve accurate results on streaming data in real time and to learn CUDA through implementation. This project successfully was able to accurately gauge the sentiment of the tweets processed in real time. Future implementations may be able to achieve the same results quicker and easier by using Pytorch or TensorFlow.

References

1. Shara Tibken. Ces 2019: Moore's law is dead, says nvidia's ceo, Jan 2019.
2. Pinkesh Badjatiya, Shashank Gupta, Manish Gupta, and Vasudeva Varma. Deep learning for hate speech detection in tweets. In *Proceedings of the 26th International Conference on World Wide Web Companion*, WWW '17 Companion, page 759–760, Republic and Canton of Geneva, CHE, 2017. International World Wide Web Conferences Steering Committee.
3. Uğur Mertoğlu and Burkay Genç. Automated fake news detection in the age of digital libraries. *Information Technology Libraries*, 39(4):1 – 19, 2020.
4. J.B. Kulkarni, A.A. Sawant, and V.S. Inamdar. Database processing by linear regression on gpu using cuda. *2011 International Conference on Signal Processing, Communication, Computing and Networking Technologies, Signal Processing, Communication, Computing and Networking Technologies (ICSCCN), 2011 International Conference on*, pages 20 – 23, 2011.
5. Alexandre Bovet and Hernán A. Makse. Influence of fake news in twitter during the 2016 us presidential election. *Nature Communications*, 10(1):7, Jan 2019.
6. Anne Cocos, Alexander G Fiks, and Aaron J Masino. Deep learning for pharmacovigilance: recurrent neural network architectures for labeling adverse drug reactions in twitter posts. *Journal of the American Medical Informatics Association : JAMIA*, 24(4):813—821, July 2017.
7. Azadeh Nikfarjam, Abeed Sarker, Karen O'Connor, Rachel Ginn, and Graciela Gonzalez. Pharmacovigilance from social media: mining adverse drug reaction mentions using sequence labeling with word embedding cluster features. *Journal of the American Medical Informatics Association*, 22(3):671–681, 03 2015.
8. Mamoon Rashid, Aamir Hamid, Nazir Ahmad, Mohammed Burhanur Rehman, and Mir Mohammad Yousuf. Novel machine learning approach for sentiment analysis of real time twitter data with apache flume. *2020 Sixth International Conference on Parallel, Distributed and Grid Computing (PDGC), Parallel, Distributed and Grid Computing (PDGC), 2020 Sixth International Conference on*, pages 336 – 340, 2020.
9. Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–1751, Doha, Qatar, October 2014. Association for Computational Linguistics.
10. Ji Young Lee and Franck Dernoncourt. Sequential short-text classification with recurrent and convolutional neural networks. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 515–520, San Diego, California, June 2016. Association for Computational Linguistics.
11. Alex Graves. Sequence transduction with recurrent neural networks, 2012.

12. Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 11 1997.
13. V Jude Nirmal and DI George Amalarethinam. Parallel implementation of big data pre-processing algorithms for sentiment analysis of social networking data. *International journal of fuzzy mathematical archive*, 6(2):149–159, 2015.
14. Michael Garland, Scott Le Grand, John Nickolls, Joshua Anderson, Jim Hardwick, Scott Morton, Everett Phillips, Yao Zhang, and Vasily Volkov. Parallel computing experiences with cuda. *IEEE Micro*, 28(4):13–27, 2008.
15. Ferhat Bozkurt, Önder Çoban, Faruk Baturalp Günay, and Şeyma Yücel Altay. High performance twitter sentiment analysis using cuda based distance kernel on gpus. *Tehnički vjesnik*, 26(5):1218–1227, 2019.
16. Siva Kumar Pathuri, N Anbazhagan, Gyanendra Prasad Joshi, and Jinsang You. Feature-based sentimental analysis on public attention towards covid-19 using cuda-sadbm classification model. *Sensors*, 22(1):80, 2022.
17. Kun Zhang, Gangming Lyu, and Xinming Luo. A deep recurrent neural network controller for nonlinear active noise control systems. *2020 IEEE 6th International Conference on Computer and Communications (ICCC), Computer and Communications (ICCC), 2020 IEEE 6th International Conference on*, pages 2393 – 2396, 2020.
18. Peter D. Turney. Thumbs up or thumbs down? *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, page 417–424, Jul 2002.

8 Users Manuel

A Organization of the Manual

The user's manual consists of 4 sections: General Information, System Summary, Getting Started, Using the System, and Reporting. The general information section explains in general terms the system and the purpose for which it is intended. The System Overview section provides a general view of the system. The Summary section outlines the uses of the system's hardware and software requirements, system's configurations, user access levels and system's behavior in case of any contingencies. The Getting Started section explains how to get the sentiment analysis program started and install it on the device. The Using the System section provides a detailed of description of system functions. The Reporting section describes in what ways information collected by the application are presented and how to access the information.

B General Information

This section explains in general terms the system and the purpose for which it is intended. The system detailed in this users manual is the compiled and packaged version of the sentiment analysis program created for the "Real Time Sentiment Analysis with CUDA" project paper. The purpose of this program is to collect tweets from Twitter and to analyze those tweets for sentiment whether

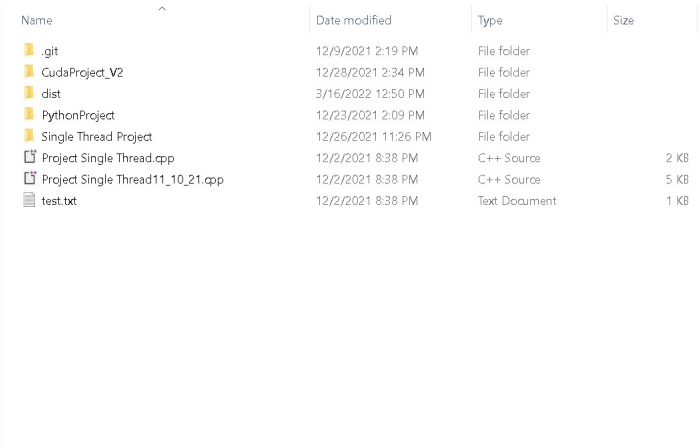
it's positive or negative. The program can be found on a cloud share drive at "https://drive.google.com/drive/folders/16WWPFvPGSq7rwvdWrZLQ-FYQ3o5b0MwH?usp=sharing". An alternative distribution of these files may be provided upon request.

C System Overview

Real-Time Sentiment Analysis is an application which allows the collecting of information via streaming Twitter data. This application only collects data from Twitter pertaining to the "Covid" metadata tag. User input to select the metadata tag may be implemented in future versions. The application's data is stored in memory so it's lost after you close out of the program. This application will open a GUI on the desktop to display results. Real-Time Sentiment Analysis operates on Windows systems only. System requirements are 1 GHz or faster processor and 2 GB of RAM. There are no additional system requirements. Real time sentiment analysis needs only read and execute permissions. Internet access is required. This application's operational status is under development.

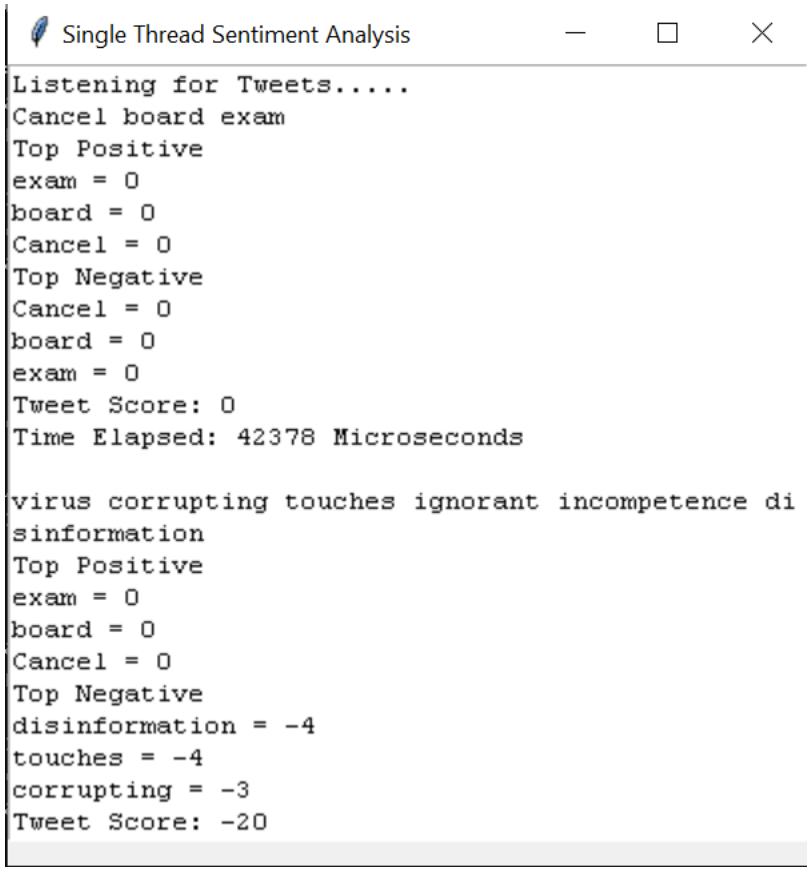
D Getting Started

In order to install the program you first need to obtain the source files from either the google drive or through other means. Once you have the compressed file you need to extract those files to a file path of your choosing. Navigate to the distribution folder and then double click on the "PythonProject.exe" executable. The program will then open up the GUI and then run. When finished, click on the "X" in the top right corner to close out the program.



Name	Date modified	Type	Size
.git	12/9/2021 2:19 PM	File folder	
CudaProject_V2	12/28/2021 2:34 PM	File folder	
dist	3/16/2022 12:50 PM	File folder	
PythonProject	12/23/2021 2:09 PM	File folder	
Single Thread Project	12/26/2021 11:26 PM	File folder	
Project Single Thread.cpp	12/2/2021 8:38 PM	C++ Source	2 KB
Project Single Thread11_10_21.cpp	12/2/2021 8:38 PM	C++ Source	5 KB
test.txt	12/2/2021 8:38 PM	Text Document	1 KB

Extract the contents of the "Sentiment Analysis.zip" folder. Double click on $\backslash SentimentAnalysis\backslash GraduateProject\backslash dist\backslash PythonProject.exe$



You should see the GUI pop up, just let it run and click the "X" when done.

E Using The System

This system collects tweets from Twitter and analyzes those tweets for sentiment using a single threaded C++ program on the back end. The tweets are collected by accessing the Twitter API and requesting tweets with the "Covid" metadata tag. In the future we hope to include user input to change the metadata tag to the user's choice. Those tweets are then parsed from JSON into a string by a Python program and piped to the C++ single threaded program for sentiment analysis. Those results are piped back into the Python program and displayed through the Python GUI. Tweets will be analyzed as they come in and the GUI will update the results. You may have to let each instance run for 10-15 minutes in order for the program to collect enough tweets to train the data with and start producing accurate results. Results displayed in the GUI are the current top three positive words collected during the current instance of the program as well as the top three negative words and their score. The tweet score is also displayed under the top thee positive and negative words. Under the tweet score

is the time elapsed from when the tweet was sent to the C++ program to when the results were sent to the Python program. There is a scroll function in the latest version of the system that allows you to scroll through past results. You may access the scroll feature by using the mouse wheel or the arrow keys on your keyboard. When you want to terminate the program just click on the "X" on the top right of the GUI. This program does not save any information and will lose all of the data upon termination.

F Reporting

This application only collects information from Twitter using the developer's Twitter account and private keys provided by the application. This information is not stored anywhere except for in memory and is not persistent or stored anywhere upon termination of the program. The collected information can be viewed through the GUI in the application. Upon termination of an instance of the program the information is no longer able the be viewed or accessed.